TUTORIAL FOUR
Process Synchronization- Part I

1. Consider the following user-level solution to the critical section problem, where **flag** and **turn** are shared variables initialized to **false** and **0**, respectively.

Process P0

```
while(1){
        flag=true;
        while(turn==1);
        critical-section
        turn=1;
        remainder-section
}
```

Process P1

```
while(1){
        turn=0;
        while(flag and turn==0);
        critical-section
        flag=false;
        remainder-section
}
```

Determine which of the three requirements in part (a) are not satisfied with justifications.

2. Indicate whether the following statements are true or false. Justify your answers.
   a) Mutual exclusion can be achieved by disabling interrupts during the critical section.
   b) If a solution to a critical section problem satisfies progress, then it also satisfies bounded waiting.

3. Consider the following implementation of **TestAndSet()**.

```
boolean TestAndSet (boolean *target) {
    boolean rv = *target;

    *target = false;

     return rv ;

}
```

Consider the following hardware-level solution to the critical section problem, where the code for a process P_i **(i = 0 or 1)** is shown. A **lock** is a Boolean variable. The initial value of **lock=True**.

```
Process Pi:

 While(1){
   while(TestAndSet(&lock)==false);

   critical-section
    lock = false;

   remainder-section

 }
```

Does this solution satisfy the properties of mutual exclusion and progress? Justify your answers.